

You can order print and ebook versions of *Think Bayes 2e* from Bookshop.org and Amazon.

Testing

```
# install empiricaldist if necessary
```

```
try:
    import empiricaldist
except ImportError:
    !pip install empiricaldist
    import empiricaldist
```

```
# Get utils.py
```

```
from os.path import basename, exists
```

```
def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve
        local, _ = urlretrieve(url, filename)
        print('Downloaded ' + local)
```

```
download('https://github.com/AllenDowney/ThinkBayes2/raw/master/soln/utils.py')
```

```
from utils import set_pyplot_params
set_pyplot_params()
```

In <<_TheEuroProblem>> I presented a problem from David MacKay's book, *Information Theory, Inference, and Learning Algorithms*:

"A statistical statement appeared in *The Guardian* on Friday January 4, 2002:

When spun on edge 250 times, a Belgian one-euro coin came up heads 140 times and tails 110. 'It looks very suspicious to me,' said Barry Blight, a statistics lecturer at the London School of Economics. 'If the coin were unbiased, the chance of getting a result as extreme as that would be less than 7%.'

"But [MacKay asks] do these data give evidence that the coin is biased rather than fair?"

We started to answer this question in <<_EstimatingProportions>>; to review, our answer was based on these modeling decisions:

- If you spin a coin on edge, there is some probability, x , that it will land heads up.
- The value of x varies from one coin to the next, depending on how the coin is balanced and possibly other factors.

Starting with a uniform prior distribution for x , we updated it with the given data, 140 heads and 110 tails. Then we used the posterior distribution to compute the most likely value of x , the posterior mean, and a credible interval.

But we never really answered MacKay's question: "Do these data give evidence that the coin is biased rather than fair?"

In this chapter, finally, we will.

Estimation

Let's review the solution to the Euro problem from <<_TheBinomialLikelihoodFunction>>. We started with a uniform prior.

```
import numpy as np
from empiricaldist import Pmf

xs = np.linspace(0, 1, 101)
uniform = Pmf(1, xs)
```

And we used the binomial distribution to compute the probability of the data for each possible value of x .

```
from scipy.stats import binom

k, n = 140, 250
likelihood = binom.pmf(k, n, xs)
```

We computed the posterior distribution in the usual way.

```
posterior = uniform * likelihood
posterior.normalize()
```

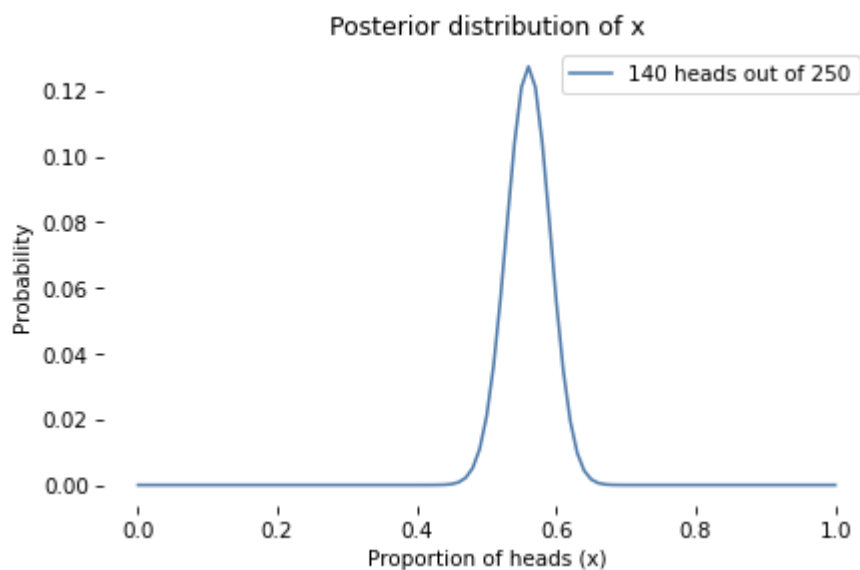
```
np.float64(0.39840637450199234)
```

And here's what it looks like.

```
from utils import decorate

posterior.plot(label='140 heads out of 250')

decorate(xlabel='Proportion of heads (x)',
         ylabel='Probability',
         title='Posterior distribution of x')
```



Again, the posterior mean is about 0.56, with a 90% credible interval from 0.51 to 0.61.

```
print(posterior.mean(),
      posterior.credible_interval(0.9))
```

```
0.5595238095238096 [0.51 0.61]
```

The prior mean was 0.5, and the posterior mean is 0.56, so it seems like the data is evidence that the coin is biased.

But, it turns out not to be that simple.

Evidence

In <<_OliversBlood>>, I said that data are considered evidence in favor of a hypothesis, A , if the data are more likely under A than under the alternative, B ; that is if

$$P(D|A) > P(D|B)$$

Furthermore, we can quantify the strength of the evidence by computing the ratio of these likelihoods, which is known as the Bayes factor and often denoted K :

$$K = \frac{P(D|A)}{P(D|B)}$$

So, for the Euro problem, let's consider two hypotheses, `fair` and `biased`, and compute the likelihood of the data under each hypothesis.

If the coin is fair, the probability of heads is 50%, and we can compute the probability of the data (140 heads out of 250 spins) using the binomial distribution:

```
k = 140
n = 250

like_fair = binom.pmf(k, n, p=0.5)
like_fair
```

```
np.float64(0.008357181724918205)
```

That's the probability of the data, given that the coin is fair.

But if the coin is biased, what's the probability of the data? That depends on what "biased" means. If we know ahead of time that "biased" means the probability of heads is 56%, we can use the binomial distribution again:

```
like_biased = binom.pmf(k, n, p=0.56)
like_biased
```

```
np.float64(0.0507781595951839)
```

Now we can compute the likelihood ratio:

```
K = like_biased / like_fair
K
```

```
np.float64(6.075990838368527)
```

The data are about 6 times more likely if the coin is biased, by this definition, than if it is fair.

But we used the data to define the hypothesis, which seems like cheating. To be fair, we should define "biased" before we see the data.

Uniformly Distributed Bias

Suppose "biased" means that the probability of heads is anything except 50%, and all other values are equally likely.

We can represent that definition by making a uniform distribution and removing 50%.

```
biased_uniform = uniform.copy()
biased_uniform[0.5] = 0
biased_uniform.normalize()
```

```
np.int64(100)
```

To compute the total probability of the data under this hypothesis, we compute the conditional probability of the data for each value of x .

```
xs = biased_uniform.qs
likelihood = binom.pmf(k, n, xs)
```

Then multiply by the prior probabilities and add up the products:

```
like_uniform = np.sum(biased_uniform * likelihood)
like_uniform
```

```
np.float64(0.003900491927770742)
```

So that's the probability of the data under the "biased uniform" hypothesis.

Now we can compute the likelihood ratio of the data under the `fair` and `biased uniform` hypotheses:

```
K = like_fair / like_uniform
K
```

```
np.float64(2.142596851801359)
```

The data are about two times more likely if the coin is fair than if it is biased, by this definition of "biased".

To get a sense of how strong that evidence is, we can apply Bayes's rule. For example, if the prior probability is 50% that the coin is biased, the prior odds are 1, so the posterior odds are about 2.1 to 1 and the posterior probability is about 68%.

```
prior_odds = 1
posterior_odds = prior_odds * K
posterior_odds
```

```
np.float64(2.142596851801359)
```

```
def prob(o):
    return o / (o+1)
```

```
posterior_probability = prob(posterior_odds)
posterior_probability
```

```
np.float64(0.6817918278551088)
```

Evidence that "moves the needle" from 50% to 68% is not very strong.

Now suppose "biased" doesn't mean every value of x is equally likely. Maybe values near 50% are more likely and values near the extremes are less likely. We could use a triangle-shaped distribution to represent this alternative definition of "biased":

```
ramp_up = np.arange(50)
ramp_down = np.arange(50, -1, -1)
a = np.append(ramp_up, ramp_down)

triangle = Pmf(a, xs, name='triangle')
triangle.normalize()
```

```
np.int64(2500)
```

As we did with the uniform distribution, we can remove 50% as a possible value of x (but it doesn't make much difference if we skip this detail).

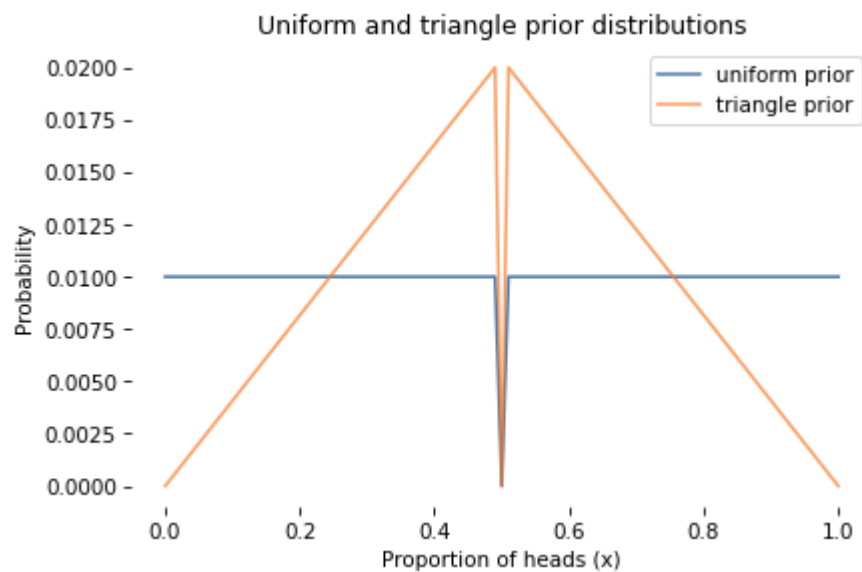
```
biased_triangle = triangle.copy()
biased_triangle[0.5] = 0
biased_triangle.normalize()
```

```
np.float64(0.98)
```

Here's what the triangle prior looks like, compared to the uniform prior.

```
biased_uniform.plot(label='uniform prior')
biased_triangle.plot(label='triangle prior')

decorate(xlabel='Proportion of heads (x)',
         ylabel='Probability',
         title='Uniform and triangle prior distributions')
```



Exercise: Now compute the total probability of the data under this definition of "biased" and compute the Bayes factor, compared with the fair hypothesis. Is the data evidence that the coin is biased?

Solution goes here

Solution goes here

Solution goes here

Bayesian Hypothesis Testing

What we've done so far in this chapter is sometimes called "Bayesian hypothesis testing" in contrast with statistical hypothesis testing.

In statistical hypothesis testing, we compute a p-value, which is hard to define concisely, and use it to determine whether the results are "statistically significant", which is also hard to define concisely.

The Bayesian alternative is to report the Bayes factor, K , which summarizes the strength of the evidence in favor of one hypothesis or the other.

Some people think it is better to report K than a posterior probability because K does not depend on a prior probability. But as we saw in this example, K often depends on a precise definition of the hypotheses, which can be just as controversial as a prior probability.

In my opinion, Bayesian hypothesis testing is better because it measures the strength of the evidence on a continuum, rather than trying to make a binary determination. But it doesn't solve what I think is the fundamental problem, which is that hypothesis testing is not asking the question we really care about.

To see why, suppose you test the coin and decide that it is biased after all. What can you do with this answer? In my opinion, not much. In contrast, there are two questions I think are more useful (and therefore more meaningful):

- Prediction: Based on what we know about the coin, what should we expect to happen in the future?
- Decision-making: Can we use those predictions to make better decisions?

At this point, we've seen a few examples of prediction. For example, in `<<_PoissonProcesses>>` we used the posterior distribution of goal-scoring rates to predict the outcome of soccer games.

And we've seen one previous example of decision analysis: In `<<_DecisionAnalysis>>` we used the distribution of prices to choose an optimal bid on *The Price is Right*.

So let's finish this chapter with another example of Bayesian decision analysis, the Bayesian Bandit strategy.

Bayesian Bandits

If you have ever been to a casino, you have probably seen a slot machine, which is sometimes called a "one-armed bandit" because it has a handle like an arm and the ability to take money like a bandit.

The Bayesian Bandit strategy is named after one-armed bandits because it solves a problem based on a simplified version of a slot machine.

Suppose that each time you play a slot machine, there is a fixed probability that you win. And suppose that different machines give you different probabilities of winning, but you don't know what the probabilities are.

Initially, you have the same prior belief about each of the machines, so you have no reason to prefer one over the others. But if you play each machine a few times, you can use the results to estimate the probabilities. And you can use the estimated probabilities to decide which machine to play next.

At a high level, that's the Bayesian bandit strategy. Now let's see the details.

Prior Beliefs

If we know nothing about the probability of winning, we can start with a uniform prior.

```
xs = np.linspace(0, 1, 101)
prior = Pmf(1, xs)
prior.normalize()
```

```
np.int64(101)
```

Supposing we are choosing from four slot machines, I'll make four copies of the prior, one for each machine.

```
beliefs = [prior.copy() for i in range(4)]
```

This function displays four distributions in a grid.

```
import matplotlib.pyplot as plt

options = dict(xticklabels='invisible', yticklabels='invisible')

def plot(beliefs, **options):
    for i, pmf in enumerate(beliefs):
        plt.subplot(2, 2, i+1)
        pmf.plot(label='Machine %s' % i)
        decorate(yticklabels=[])

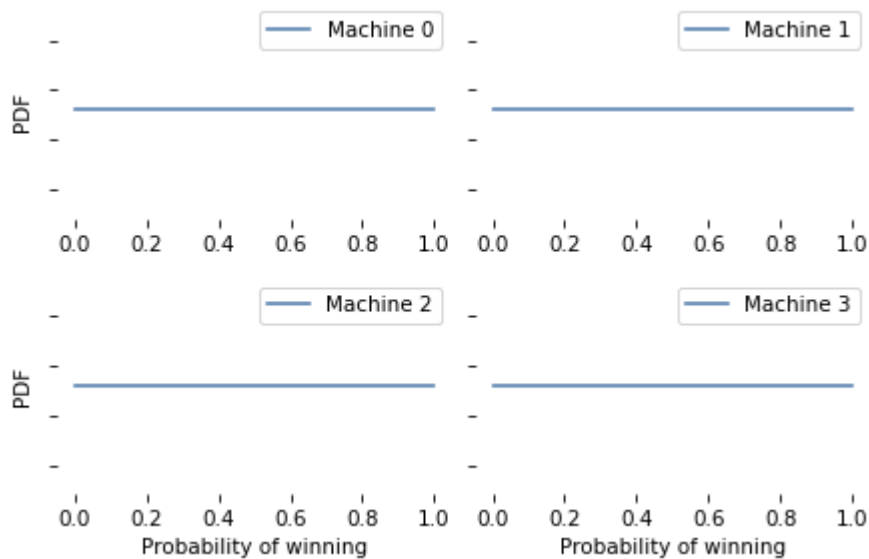
        if i in [0, 2]:
            decorate(ylabel='PDF')

        if i in [2, 3]:
            decorate(xlabel='Probability of winning')

    plt.tight_layout()
```

Here's what the prior distributions look like for the four machines.

```
plot(beliefs)
```



The Update

Each time we play a machine, we can use the outcome to update our beliefs. The following function does the update.

```
likelihood = {
    'W': xs,
    'L': 1 - xs
}
```

```
def update(pmf, data):
    """Update the probability of winning."""
    pmf *= likelihood[data]
    pmf.normalize()
```

This function updates the prior distribution in place. `pmf` is a `Pmf` that represents the prior distribution of `x`, which is the probability of winning.

`data` is a string, either `W` if the outcome is a win or `L` if the outcome is a loss.

The likelihood of the data is either `xs` or `1-xs`, depending on the outcome.

Suppose we choose a machine, play 10 times, and win once. We can compute the posterior distribution of `x`, based on this outcome, like this:

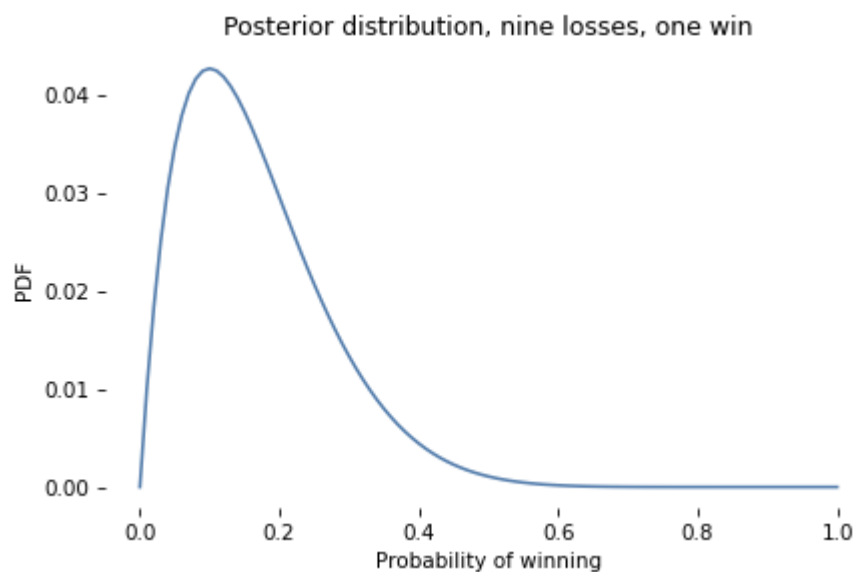
```
np.random.seed(17)
```

```
bandit = prior.copy()

for outcome in 'WLLLLLLLLL':
    update(bandit, outcome)
```

Here's what the posterior looks like.

```
bandit.plot()
decorate(xlabel='Probability of winning',
        ylabel='PDF',
        title='Posterior distribution, nine losses, one win')
```



Multiple Bandits

Now suppose we have four machines with these probabilities:

```
actual_probs = [0.10, 0.20, 0.30, 0.40]
```

Remember that as a player, we don't know these probabilities.

The following function takes the index of a machine, simulates playing the machine once, and returns the outcome, `W` or `L`.

```
from collections import Counter

# count how many times we've played each machine
counter = Counter()

def play(i):
    """Play machine i.

    i: index of the machine to play

    returns: string 'W' or 'L'
    """
    counter[i] += 1
    p = actual_probs[i]
    if np.random.random() < p:
        return 'W'
    else:
        return 'L'
```

`counter` is a `Counter`, which is a kind of dictionary we'll use to keep track of how many times each machine is played.

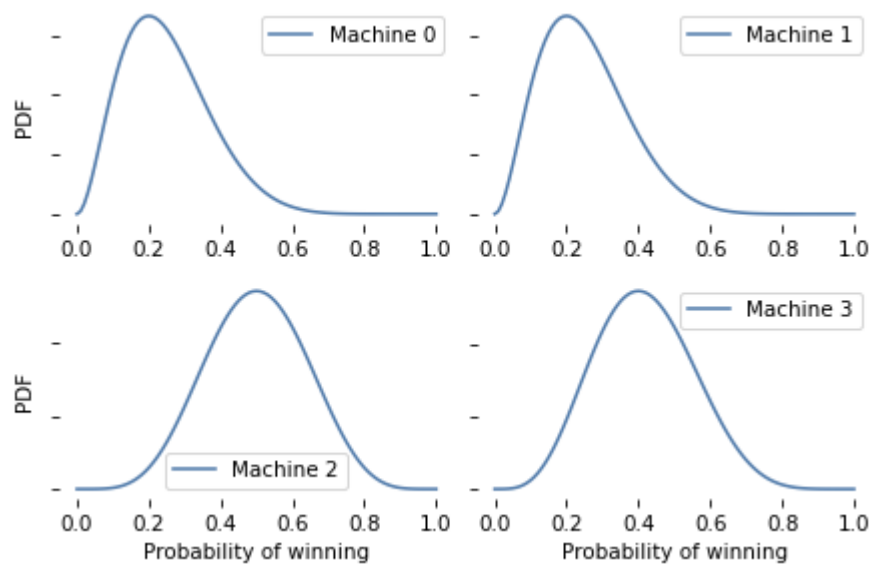
Here's a test that plays each machine 10 times.

```
for i in range(4):
    for _ in range(10):
        outcome = play(i)
        update(beliefs[i], outcome)
```

Each time through the inner loop, we play one machine and update our beliefs.

Here's what our posterior beliefs look like.

```
plot(beliefs)
```



Here are the actual probabilities, posterior means, and 90% credible intervals.

```
import pandas as pd

def summarize_beliefs(beliefs):
    """Compute means and credible intervals.

    beliefs: sequence of Pmf

    returns: DataFrame
    """
    columns = ['Actual P(win)',
               'Posterior mean',
               'Credible interval']

    df = pd.DataFrame(columns=columns)
    for i, b in enumerate(beliefs):
        mean = np.round(b.mean(), 3)
        ci = b.credible_interval(0.9)
        ci = np.round(ci, 3)
        df.loc[i] = actual_probs[i], mean, ci
    return df
```

```
summarize_beliefs(beliefs)
```

	Actual P(win)	Posterior mean	Credible interval
0	0.1	0.250	[0.08, 0.47]
1	0.2	0.250	[0.08, 0.47]
2	0.3	0.500	[0.27, 0.73]
3	0.4	0.417	[0.2, 0.65]

We expect the credible intervals to contain the actual probabilities most of the time.

Explore and Exploit

Based on these posterior distributions, which machine do you think we should play next? One option would be to choose the machine with the highest posterior mean.

That would not be a bad idea, but it has a drawback: since we have only played each machine a few times, the posterior distributions are wide and overlapping, which means we are not sure which machine is the best; if we focus on one machine too soon, we might choose the wrong machine and play it more than we should.

To avoid that problem, we could go to the other extreme and play all machines equally until we are confident we have identified the best machine, and then play it exclusively.

That's not a bad idea either, but it has a drawback: while we are gathering data, we are not making good use of it; until we're sure which machine is the best, we are playing the others more than we should.

The Bayesian Bandits strategy avoids both drawbacks by gathering and using data at the same time. In other words, it balances exploration and exploitation.

The kernel of the idea is called Thompson sampling: when we choose a machine, we choose at random so that the probability of choosing each machine is proportional to the probability that it is the best.

Given the posterior distributions, we can compute the "probability of superiority" for each machine.

Here's one way to do it. We can draw a sample of 1000 values from each posterior distribution, like this:

```
samples = np.array([b.choice(1000)
                    for b in beliefs])
samples.shape
```

```
(4, 1000)
```

The result has 4 rows and 1000 columns. We can use `argmax` to find the index of the largest value in each column:

```
indices = np.argmax(samples, axis=0)
indices.shape
```

```
(1000,)
```

The `Pmf` of these indices is the fraction of times each machine yielded the highest values.

```
pmf = Pmf.from_seq(indices)
pmf
```

	probs
0	0.048
1	0.043
2	0.625
3	0.284

These fractions approximate the probability of superiority for each machine. So we could choose the next machine by choosing a value from this `Pmf`.

```
pmf.choice()
```

```
array([1])
```

But that's a lot of work to choose a single value, and it's not really necessary, because there's a shortcut.

If we draw a single random value from each posterior distribution and select the machine that yields the highest value, it turns out that we'll select each machine in proportion to its probability of superiority.

That's what the following function does.

```
def choose(beliefs):
    """Use Thompson sampling to choose a machine.

    Draws a single sample from each distribution.

    returns: index of the machine that yielded the highest value
    """
    ps = [b.choice() for b in beliefs]
    return np.argmax(ps)
```

This function chooses one value from the posterior distribution of each machine and then uses `argmax` to find the index of the machine that yielded the highest value.

Here's an example.

```
choose(beliefs)
```

```
np.int64(3)
```

The Strategy

Putting it all together, the following function chooses a machine, plays once, and updates `beliefs` :

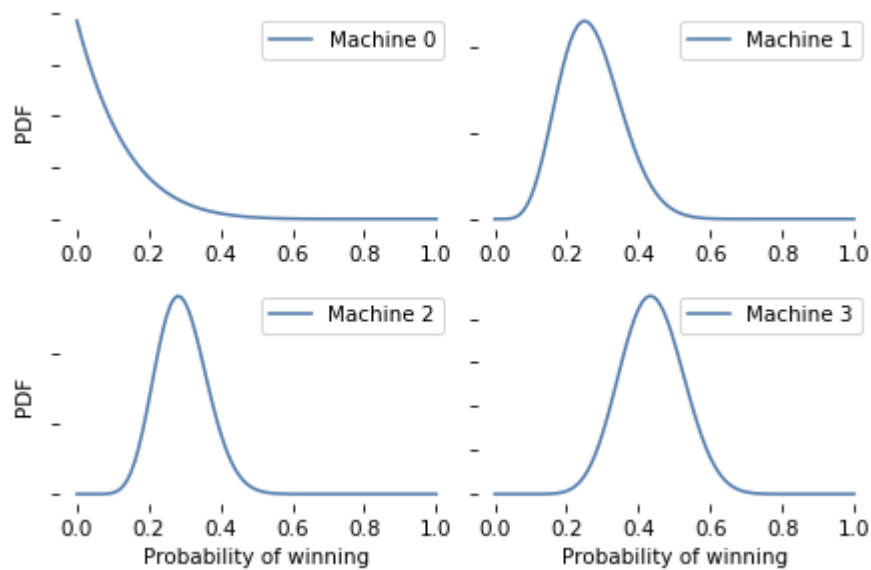
```
def choose_play_update(beliefs):  
    """Choose a machine, play it, and update beliefs."""  
  
    # choose a machine  
    machine = choose(beliefs)  
  
    # play it  
    outcome = play(machine)  
  
    # update beliefs  
    update(beliefs[machine], outcome)
```

To test it out, let's start again with a fresh set of beliefs and an empty `Counter` .

```
beliefs = [prior.copy() for i in range(4)]  
counter = Counter()
```

If we run the bandit algorithm 100 times, we can see how `beliefs` gets updated:

```
num_plays = 100  
  
for i in range(num_plays):  
    choose_play_update(beliefs)  
  
plot(beliefs)
```

The following table summarizes the results.

```
summarize_beliefs(beliefs)
```

	Actual P(win)	Posterior mean	Credible interval
0	0.1	0.107	[0.0, 0.31]
1	0.2	0.269	[0.14, 0.42]
2	0.3	0.293	[0.18, 0.41]
3	0.4	0.438	[0.3, 0.58]

The credible intervals usually contain the actual probabilities of winning. The estimates are still rough, especially for the lower-probability machines. But that's a feature, not a bug: the goal is to play the high-probability machines most often. Making the estimates more precise is a means to that end, but not an end itself.

More importantly, let's see how many times each machine got played.

```
def summarize_counter(counter):  
    """Report the number of times each machine was played.  
  
    counter: Collections.Counter  
  
    returns: DataFrame  
    """  
    index = range(4)  
    columns = ['Actual P(win)', 'Times played']  
    df = pd.DataFrame(index=index, columns=columns)  
    for i, count in counter.items():  
        df.loc[i] = actual_probs[i], count  
    return df
```

```
summarize_counter(counter)
```

	Actual P(win)	Times played
0	0.1	7
1	0.2	24
2	0.3	39
3	0.4	30

If things go according to plan, the machines with higher probabilities should get played more often.

Summary

In this chapter we finally solved the Euro problem, determining whether the data support the hypothesis that the coin is fair or biased. We found that the answer depends on how we define "biased". And we summarized the results using a Bayes factor, which quantifies the strength of the evidence.

But the answer wasn't satisfying because, in my opinion, the question wasn't interesting. Knowing whether the coin is biased is not useful unless it helps us make better predictions and better decisions.

As an example of a more interesting question, we looked at the "one-armed bandit" problem and a strategy for solving it, the Bayesian bandit algorithm, which tries to balance exploration and exploitation, that is, gathering more information and making the best use of the information we have.

As an exercise, you'll have a chance to explore adaptive strategies for standardized testing.

Bayesian bandits and adaptive testing are examples of Bayesian decision theory, which is the idea of using a posterior distribution as part of a decision-making process, often by choosing an action that minimizes the costs we expect on average (or maximizes a benefit).

The strategy we used in `<<_MaximizingExpectedGain>>` to bid on *The Price is Right* is another example.

These strategies demonstrate what I think is the biggest advantage of Bayesian methods over classical statistics. When we represent knowledge in the form of probability distributions, Bayes's theorem tells us how to change our beliefs as we get more data, and Bayesian decision theory tells us how to make that knowledge actionable.

Exercises

Exercise: Standardized tests like the SAT are often used as part of the admission process at colleges and universities. The goal of the SAT is to measure the academic preparation of the test-takers; if it is accurate, their scores should reflect their actual ability in the domain of the test.

Until recently, tests like the SAT were taken with paper and pencil, but now students have the option of taking the test online. In the online format, it is possible for the test to be "adaptive", which means that it can choose each question based on responses to previous questions.

If a student gets the first few questions right, the test can challenge them with harder questions. If they are struggling, it can give them easier questions. Adaptive testing has the potential to be more "efficient", meaning that with the same number of questions an adaptive test could measure the ability of a tester more precisely.

To see whether this is true, we will develop a model of an adaptive test and quantify the precision of its measurements.

Details of this exercise are in the notebook.

The Model

The model we'll use is based on item response theory, which assumes that we can quantify the difficulty of each question and the ability of each test-taker, and that the probability of a correct response is a function of difficulty and ability.

Specifically, a common assumption is that this function is a three-parameter logistic function:

$$p = c + \frac{1 - c}{1 + e^{-a(\theta - b)}}$$

where θ is the ability of the test-taker and b is the difficulty of the question.

c is the lowest probability of getting a question right, supposing the test-taker with the lowest ability tries to answer the hardest question. On a multiple-choice test with four responses, c might be 0.25, which is the probability of getting the right answer by guessing at random.

a controls the shape of the curve.

The following function computes the probability of a correct answer, given `ability` and `difficulty`:

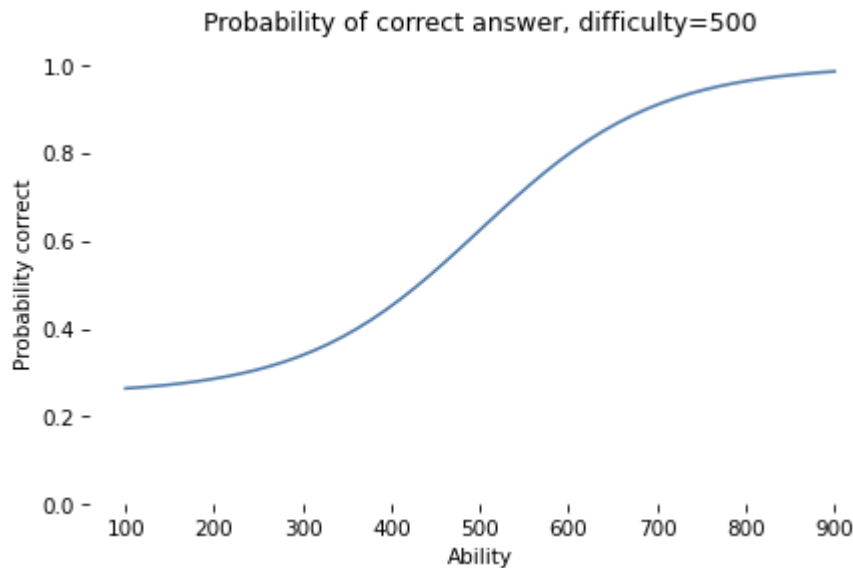
```
def prob_correct(ability, difficulty):
    """Probability of a correct response."""
    a = 100
    c = 0.25
    x = (ability - difficulty) / a
    p = c + (1-c) / (1 + np.exp(-x))
    return p
```

I chose `a` to make the range of scores comparable to the SAT, which reports scores from 200 to 800.

Here's what the logistic curve looks like for a question with difficulty 500 and a range of abilities.

```
abilities = np.linspace(100, 900)
diff = 500
ps = prob_correct(abilities, diff)
```

```
plt.plot(abilities, ps)
decorate(xlabel='Ability',
        ylabel='Probability correct',
        title='Probability of correct answer, difficulty=500',
        ylim=[0, 1.05])
```



Someone with `ability=900` is nearly certain to get the right answer. Someone with `ability=100` has about a 25% chance of getting the right answer by guessing.

Simulating the Test

To simulate the test, we'll use the same structure we used for the bandit strategy:

- A function called `play` that simulates a test-taker answering one question.
- A function called `choose` that chooses the next question to pose.
- A function called `update` that uses the outcome (a correct response or not) to update the estimate of the test-taker's ability.

Here's `play`, which takes `ability` and `difficulty` as parameters.

```
def play(ability, difficulty):
    """Simulate a test-taker answering a question."""
    p = prob_correct(ability, difficulty)
    return np.random.random() < p
```

`play` uses `prob_correct` to compute the probability of a correct answer and `np.random.random` to generate a random value between 0 and 1. The return value is `True` for a correct response and `False` otherwise.

As a test, let's simulate a test-taker with `ability=600` answering a question with `difficulty=500`. The probability of a correct response is about 80%.

```
prob_correct(600, 500)
```

```
np.float64(0.7982939339725037)
```

Suppose this person takes a test with 51 questions, all with the same difficulty, 500. We expect them to get about 80% of the questions correct.

Here's the result of one simulation.

```
np.random.seed(18)
```

```
num_questions = 51
outcomes = [play(600, 500) for _ in range(num_questions)]
np.mean(outcomes)
```

```
np.float64(0.803921568627451)
```

We expect them to get about 80% of the questions right.

Now let's suppose we don't know the test-taker's ability. We can use the data we just generated to estimate it. And that's what we'll do next.

The Prior

The SAT is designed so the distribution of scores is roughly normal, with mean 500 and standard deviation 100. So the lowest score, 200, is three standard deviations below the mean, and the highest score, 800, is three standard deviations above.

We could use that distribution as a prior, but it would tend to cut off the low and high ends of the distribution. Instead, I'll inflate the standard deviation to 300, to leave open the possibility that ability can be less than 200 or more than 800.

Here's a Pmf that represents the prior distribution.

```
from scipy.stats import norm

mean = 500
std = 300

qs = np.linspace(0, 1000)
ps = norm(mean, std).pdf(qs)

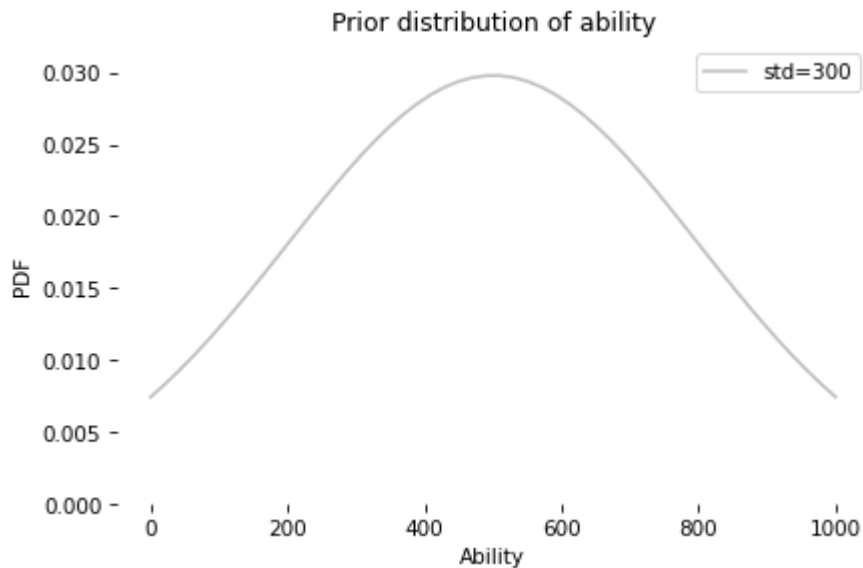
prior = Pmf(ps, qs)
prior.normalize()
```

```
np.float64(0.04464186995102338)
```

And here's what it looks like.

```
prior.plot(label='std=300', color='C5')

decorate(xlabel='Ability',
         ylabel='PDF',
         title='Prior distribution of ability',
         ylim=[0, 0.032])
```



The Update

The following function takes a prior `Pmf` and the outcome of a single question, and updates the `Pmf` in place.

```
def update_ability(pmf, data):
    """Update the distribution of ability."""
    difficulty, outcome = data

    abilities = pmf.qs
    ps = probab_correct(abilities, difficulty)

    if outcome:
        pmf *= ps
    else:
        pmf *= 1 - ps

    pmf.normalize()
```

`data` is a tuple that contains the difficulty of a question and the outcome: `True` if the response was correct and `False` otherwise.

As a test, let's do an update based on the outcomes we simulated previously, based on a person with `ability=600` answering 51 questions with `difficulty=500`.

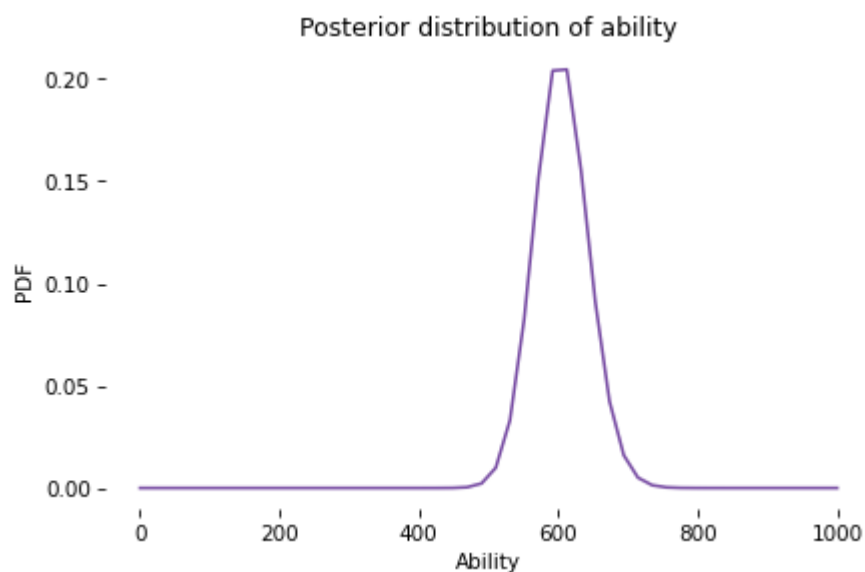
```
actual_600 = prior.copy()

for outcome in outcomes:
    data = (500, outcome)
    update_ability(actual_600, data)
```

Here's what the posterior distribution looks like.

```
actual_600.plot(color='C4')

decorate(xlabel='Ability',
        ylabel='PDF',
        title='Posterior distribution of ability')
```



The posterior mean is pretty close to the test-taker's actual ability, which is 600.

```
actual_600.mean()
```

```
np.float64(604.3325737356816)
```

If we run this simulation again, we'll get different results.

Adaptation

Now let's simulate an adaptive test. I'll use the following function to choose questions, starting with the simplest strategy: all questions have the same difficulty.


```
def choose(i, belief):
    """Choose the difficulty of the next question."""
    return 500
```

As parameters, `choose` takes `i`, which is the index of the question, and `belief`, which is a `Pmf` representing the posterior distribution of `ability`, based on responses to previous questions.

This version of `choose` doesn't use these parameters; they are there so we can test other strategies (see the exercises at the end of the chapter).

The following function simulates a person taking a test, given that we know their actual ability.

```
def simulate_test(actual_ability):
    """Simulate a person taking a test."""
    belief = prior.copy()
    trace = pd.DataFrame(columns=['difficulty', 'outcome'])

    for i in range(num_questions):
        difficulty = choose(i, belief)
        outcome = play(actual_ability, difficulty)
        data = (difficulty, outcome)
        update_ability(belief, data)
        trace.loc[i] = difficulty, outcome

    return belief, trace
```

The return values are a `Pmf` representing the posterior distribution of ability and a `DataFrame` containing the difficulty of the questions and the outcomes.

Here's an example, again for a test-taker with `ability=600`.

```
belief, trace = simulate_test(600)
```

We can use the trace to see how many responses were correct.

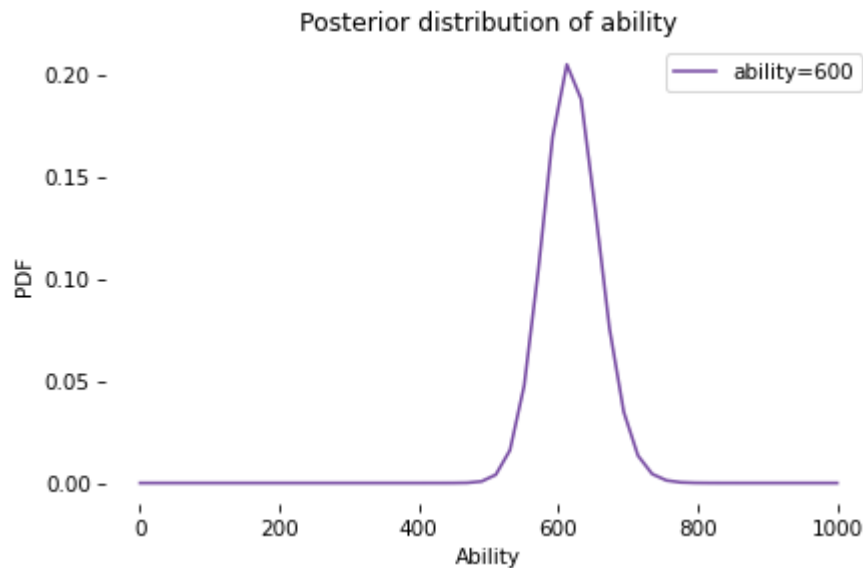
```
trace['outcome'].sum()
```

```
np.int64(42)
```

And here's what the posterior looks like.

```
belief.plot(color='C4', label='ability=600')

decorate(xlabel='Ability',
         ylabel='PDF',
         title='Posterior distribution of ability')
```



Again, the posterior distribution represents a pretty good estimate of the test-taker's actual ability.

Quantifying Precision

To quantify the precision of the estimates, I'll use the standard deviation of the posterior distribution. The standard deviation measures the spread of the distribution, so higher value indicates more uncertainty about the ability of the test-taker.

In the previous example, the standard deviation of the posterior distribution is about 40.

```
belief.mean(), belief.std()
```

```
(np.float64(618.6942050450824), np.float64(40.08554296596485))
```

For an exam where all questions have the same difficulty, the precision of the estimate depends strongly on the ability of the test-taker. To show that, I'll loop through a range of abilities and simulate a test using the version of `choice` that always returns `difficulty=500`.

```
actual_abilities = np.linspace(200, 800)
results = pd.DataFrame(columns=['ability', 'posterior_std'])
series = pd.Series(index=actual_abilities, dtype=float, name='std')

for actual_ability in actual_abilities:
    belief, trace = simulate_test(actual_ability)
    series[actual_ability] = belief.std()
```

The following plot shows the standard deviation of the posterior distribution for one simulation at each level of ability.

The results are noisy, so I also plot a curve fitted to the data by local regression.

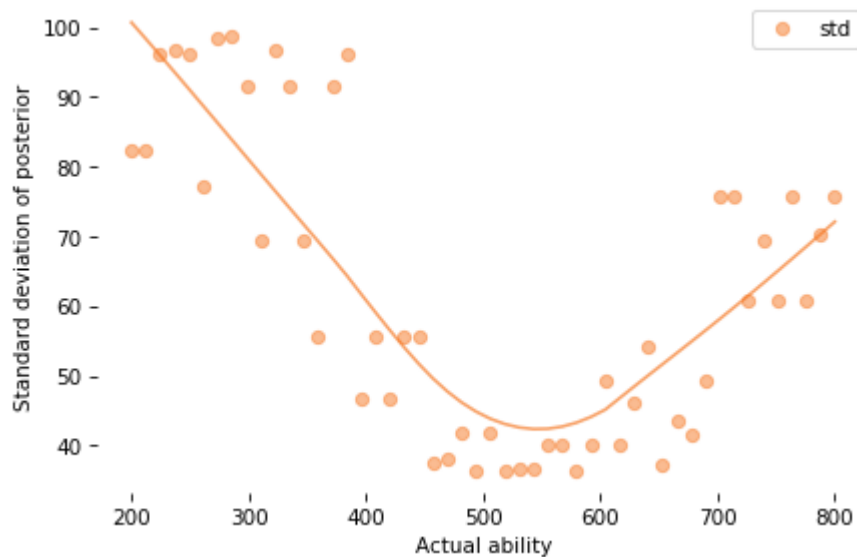
```

from utils import plot_series_lowess

plot_series_lowess(series, 'C1')

decorate(xlabel='Actual ability',
         ylabel='Standard deviation of posterior')

```



The test is most precise for people with ability between 500 and 600, less precise for people at the high end of the range, and even worse for people at the low end.

When all the questions have difficulty 500, a person with ability=800 has a high probability of getting them right. So when they do, we don't learn very much about them.

If the test includes questions with a range of difficulty, it provides more information about people at the high and low ends of the range.

As an exercise at the end of the chapter, you'll have a chance to try out other strategies, including adaptive strategies that choose each question based on previous outcomes.

Discriminatory Power

In the previous section we used the standard deviation of the posterior distribution to quantify the precision of the estimates. Another way to describe the performance of the test (as opposed to the performance of the test-takers) is to measure "discriminatory power", which is the ability of the test to distinguish correctly between test-takers with different ability.

To measure discriminatory power, I'll simulate a person taking the test 100 times; after each simulation, I'll use the mean of the posterior distribution as their "score".

```
def sample_posterior(actual_ability, iters):  
    """Simulate multiple tests and compute posterior means.  
  
    actual_ability: number  
    iters: number of simulated tests  
  
    returns: array of scores  
    """  
    scores = []  
  
    for i in range(iters):  
        belief, trace = simulate_test(actual_ability)  
        score = belief.mean()  
        scores.append(score)  
  
    return np.array(scores)
```

Here are samples of scores for people with several levels of ability.

```
sample_500 = sample_posterior(500, iters=100)
```

```
sample_600 = sample_posterior(600, iters=100)
```

```
sample_700 = sample_posterior(700, iters=100)
```

```
sample_800 = sample_posterior(800, iters=100)
```

Here's what the distributions of scores look like.

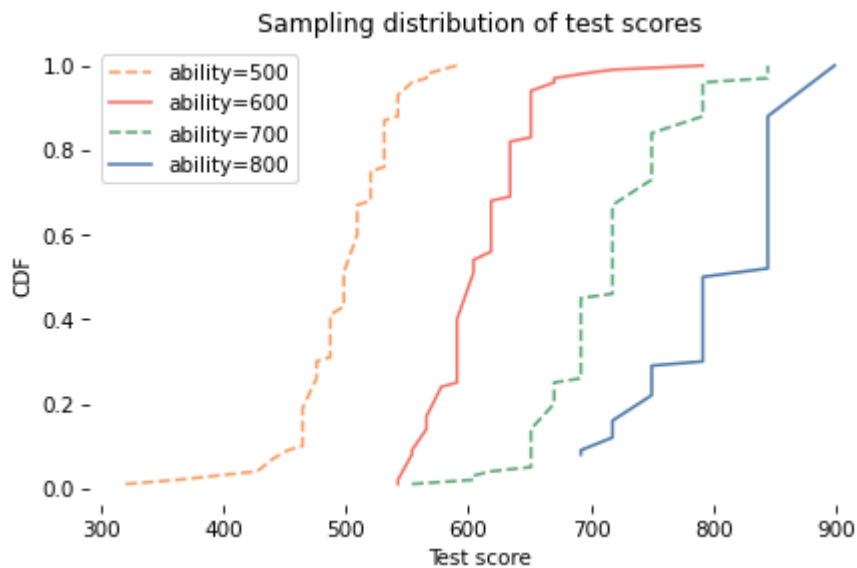
```
from empiricaldist import Cdf  
  
cdf_500 = Cdf.from_seq(sample_500)  
cdf_600 = Cdf.from_seq(sample_600)  
cdf_700 = Cdf.from_seq(sample_700)  
cdf_800 = Cdf.from_seq(sample_800)
```

```

cdf_500.plot(label='ability=500', color='C1',
             linestyle='dashed')
cdf_600.plot(label='ability=600', color='C3')
cdf_700.plot(label='ability=700', color='C2',
             linestyle='dashed')
cdf_800.plot(label='ability=800', color='C0')

decorate(xlabel='Test score',
         ylabel='CDF',
         title='Sampling distribution of test scores')

```



On average, people with higher ability get higher scores, but anyone can have a bad day, or a good day, so there is some overlap between the distributions.

For people with ability between 500 and 600, where the precision of the test is highest, the discriminatory power of the test is also high.

If people with abilities 500 and 600 take the test, it is almost certain that the person with higher ability will get a higher score.

```
np.mean(sample_600 > sample_500)
```

```
np.float64(0.98)
```

Between people with abilities 600 and 700, it is less certain.

```
np.mean(sample_700 > sample_600)
```

```
np.float64(0.95)
```

And between people with abilities 700 and 800, it is not certain at all.

```
np.mean(sample_800 > sample_700)
```

```
np.float64(0.85)
```

But remember that these results are based on a test where all questions are equally difficult. If you do the exercises at the end of the chapter, you'll see that the performance of the test is better if it includes questions with a range of difficulties, and even better if the test it is adaptive.

Go back and modify `choose`, which is the function that chooses the difficulty of the next question.

1. Write a version of `choose` that returns a range of difficulties by using `i` as an index into a sequence of difficulties.
2. Write a version of `choose` that is adaptive, so it choose the difficulty of the next question based `belief`, which is the posterior distribution of the test-taker's ability, based on the outcome of previous responses.

For both new versions, run the simulations again to quantify the precision of the test and its discriminatory power.

For the first version of `choose`, what is the ideal distribution of difficulties?

For the second version, what is the adaptive strategy that maximizes the precision of the test over the range of abilities?

```
# Solution goes here
```

```
# Solution goes here
```

Think Bayes, Second Edition

Copyright 2020 Allen B. Downey

License: Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)